

result.

`lmbench3` focussed on extending `lmbench`'s functionality along two dimensions: measuring multi-processor scalability and measuring basic aspects of processor micro-architecture.

An important feature of multi-processor systems is their ability to scale their performance. While `lmbench1` and `lmbench2` measure various important aspects of system performance, they cannot measure performance with more than one client process active at a time. Consequently, measuring performance of multi-processor and clustered systems as a function of scalable load was impossible using those tools.

`lmbench3` took the ideas and techniques developed in the earlier versions and extended them to create a new timing harness which can measure system performance under parallel, scalable loads.

`lmbench3` also includes a version of John McCalpin's STREAM benchmarks. Essentially the STREAM kernels were placed in the new `lmbench` timing harness. Since the new timing harness also measures scalability under parallel load, the `lmbench3` STREAM benchmarks include this capability automatically.

Finally, `lmbench3` includes a number of new benchmarks which measure various aspects of the processor architecture, such as basic operation latency and parallelism, to provide developers with a better understanding of system capabilities. The hope is that better informed developers will be able to better design and evaluate performance critical software in light of their increased understanding of basic system performance.

## 2. Prior Work

Benchmarking is not a new field of endeavor. There are a wide variety of approaches to benchmarking, many of which differ greatly from that taken by `lmbench`.

One common form of benchmark is to take an important application or application and workload, and to measure the time required to complete the entire task. This approach is particularly useful when evaluating the utility of systems for a single and well-known task.

Other benchmarks, such as SPECint, use a variation on this approach by measuring several applications and combining the results to predict overall performance. SPECint96 [22] extends this approach to the parallel and distributed domain by measuring the performance of a selected parallel applications built on top of MPI and/or PVM.

Another variation takes the "kernel" of an important application and measures its performance, where the "kernel" is usually a simplification of the most expensive portion of a program. Dhrystone [23] is an example of this type of benchmark as it measures the performance of important matrix operations and was

often used to predict system performance for numerical operations.

Banga developed a benchmark to measure HTTP server performance which can accurately measure server performance under high load[1]. Due to the idiosyncracies of the HTTP protocol and TCP design and implementation, there are generally operating system limits on the rate at which a single system can generate independent HTTP requests. However, Banga developed a system which can scalably present load to HTTP servers in spite of this limitation[2].

John McCalpin's STREAM benchmark measures memory bandwidth during four common vector operations[11]. It does not measure memory latency, and strictly speaking it does not measure raw memory bandwidth although memory bandwidth is crucial to STREAM performance. More recently, STREAM has been extended to measure distributed application performance using MPI to measure scalable memory subsystem performance, particularly for multi-processor machines.

Prestor[16] and Saavedra[17] have developed benchmarks which analyze memory subsystem performance.

Micro-benchmarking extends the "kernel" approach, by measuring the performance of operations or resources in isolation. `lmbench` and many other benchmarks, such as `nfsstone`[20], measure the performance of key operations so users can predict performance for certain workloads and applications by combining the performance of these operations in the right mixture.

Saavedra[18] takes the micro-benchmark approach and applies it to the problem of predicting application performance. They analyze applications or other benchmarks in terms of their "narrow spectrum benchmarks" to create a linear model of the application's computing requirements. They then measure the computer system's performance across this set of micro-benchmarks and use a linear model to predict the application's performance on the computer system. Seltzer[19] applied this technique using the features measured by `lmbench` as the basis for application prediction.

Benchmarking I/O systems has proven particularly troublesome over the years, largely due to the strong non-linearities exhibited by disk systems. Sequential I/O provides much higher bandwidth than non-sequential I/O, so performance is highly dependent on the workload characteristics as well as the file system's ability to capitalize on available sequentiality by laying out data contiguously on disk.

I/O benchmarks have a tendency to age poorly. For example, `IOStone`[15], `IOBench`[24], and the Andrew benchmark[8] used fixed size datasets, whose size was significant at the time, but which no longer measure I/O performance as the data can now fit in the

processor cache of many modern machines.

The Andrew benchmark attempts to separately measure the time to create, write, re-read, and then delete a large number of files in a hierarchical file system.

Bonnie[3] measures sequential, streaming I/O bandwidth for a single process, and random I/O latency for multiple processes.

Peter Chen developed an adaptive harness for I/O benchmarking[6][5], which defines I/O load in terms of five parameters, `uniqueBytes`, `sizeMean`, `readFrac`, `seqFrac`, and `processNum`. The benchmark then explores the parameter space to measure file system performance in a scalable fashion.

Parkbench[14] is a benchmark suite that can analyze parallel and distributed computer performance. It contains a variety of benchmarks that measure both aspects of system performance, such as communication overheads, and distributed application kernel performance. Parkbench contains benchmarks from both NAS[13] and Genesis[7].

### 3. Timing Harness

The first, and most crucial element in extending `lmbench2` so that it could measure scalable performance, was to develop a new timing harness that could accurately measure performance for any given load. Once this was done, then each benchmark would be migrated to the new timing harness.

The harness is designed to accomplish a number of goals:

1. during any timing interval of any child it is guaranteed that all other child processes are also running the benchmark
2. the timing intervals are long enough to average out most transient OS scheduler affects
3. the timing intervals are long enough to ensure that error due to clock resolution is negligible
4. timing measurements can be postponed to allow the OS scheduler to settle and adjust to the load
5. the reported results should be representative and the data analysis should be robust
6. timing intervals should be as short as possible while ensuring accurate results

Developing an accurate timing harness with a valid experimental design is more difficult than is generally supposed. Many programs incorporate elementary timing harnesses which may suffer from one or more defects, such as insufficient care taken to ensure that the benchmarked operation is run long enough to ensure that the error introduced by the clock resolution is insignificant. The basic elements of a good timing harness are discussed in Staelin[21].

The new timing harness must also collect and process the timing results from all the child processes so that it can report the representative performance. It currently

reports the median performance over all timing intervals from all child processes. It might perhaps be argued that it should report the median of the medians.

When running benchmarks with more than one child, the harness must first get a baseline estimate of performance by running the benchmark in only one process using the standard `lmbench` timing interval, which is often 5,000 microseconds. Using this information, the harness can compute the average time per iteration for a single process, and it uses this figure to compute the number of iterations necessary to ensure that each child runs for at least one second.

#### 3.1. Clock resolution

`lmbench` uses the `gettimeofday` clock, whose interface resolves time down to 1 microsecond. However, many system clock's resolution is only 10 milliseconds, and there is no portable way to query the system to discover the true clock resolution.

The problem is that the timing intervals must be substantially larger than the clock resolution in order to ensure that the timing error doesn't impact the results. For example, the true duration of an event measured with a 10 milli-second clock can vary  $\pm 10$  milliseconds from the true time, assuming that the reported time is always a truncated version of the true time. If the clock itself is not updated precisely, the true error can be even larger. This implies that timing intervals on these systems should be at least 1 second.

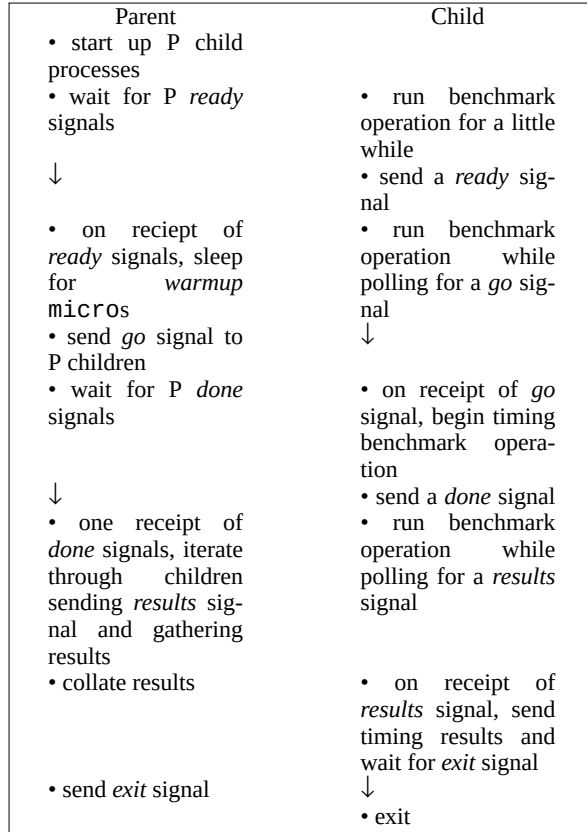
However, the `gettimeofday` clock resolution in most modern systems is 1 microsecond, so timing intervals can as small as a few milli-seconds without incurring significant timing errors related to clock resolution.

Since there is no standard interface to query the operating system for the clock resolution, `lmbench` must experimentally determine the appropriate timing interval duration which provides results in a timely fashion with a negligible clock resolution error.

#### 3.2. Coordination

Developing a timing harness that correctly manages  $N$  processes and accurately measures system performance over those same  $N$  processes is significantly more difficult than simply measuring system performance with a single process because of the asynchronous nature of parallel programming.

In essence, the new timing harness needs to create  $N$  jobs, and measure the average performance of the target subsystem while all  $N$  jobs are running. This is a standard problem for parallel and distributed programming, and involves starting the child processes and then stepping through a handshaking process to ensure that all children have started executing the benchmarked operation before any child starts taking measurements.



**Table 1. Timing harness sequencing**

Table 1 shows how the parent and child processes coordinate their activities to ensure that all children are actively running the benchmark activity while any child could be taking timing measurements.

The reason for the separate "exit" signal is to ensure that all properly managed children are alive until the parent allows them to die. This means that any SIGCHLD events that occur before the "exit" signal indicate a child failure.

### 3.3. Accuracy

The new timing harness also needs to ensure that the timing intervals are long enough for the results to be representative. The previous timing harness assumed that only single process results were important, and it was able to use timing intervals as short as possible while ensuring that errors introduced by the clock resolution were negligible. In many instances this meant that the timing intervals were smaller than a single scheduler time slice. The new timing harness must run benchmarked operations long enough to ensure that timing intervals are longer than a single scheduler time slice. Otherwise, you can get results which are complete nonsense. For example, running several copies of an `lmbench2` benchmark on a uni-processor machine will often report that the per-process performance with  $N$  jobs running in parallel is equivalent

to the performance with a single job running!<sup>1</sup>

In addition, since the timing intervals now have to be longer than a single scheduler time slice, they also need to be long enough so that a single scheduler time slice is insignificant compared to the timing interval. Otherwise the timing results can be dramatically affected by small variations in the scheduler's behavior.

Currently `lmbench` does not measure the scheduler timeslice; the design blithely assumes that timeslices are generally on the order of 10-20ms, so one second timing intervals are sufficient. Some schedulers may utilize longer time slices, but this has not (yet) been a problem.

### 3.4. Resource consumption

One important design goal was that resource consumption be constant with respect to the number of child processes. This is why the harness uses shared pipes to communicate with the children, rather than having a separate set of pipes to communicate with each child. An early design of the system utilized a pair of pipes per child for communication and synchronization between the master and slave processes. However, as the number of child processes grew, the fraction of system resources consumed by the harness grew and the additional system overhead could start to interfere with the accuracy of the measurements.

Additionally, if the master has to poll (`select`)  $N$  pipes, then the system overhead of that operation also scales with the number of children.

### 3.5. Pipe atomicity

Since all communication between the master process and the slave (child) processes is done via a set of shared pipes, we have to ensure that we never have a situation where the message can be garbled by the intermingling of two separate messages from two separate children. This is ensured by either using pipe operations that are guaranteed to be atomic on all machines, or by coordinating between processes so that at most one process is writing at a time.

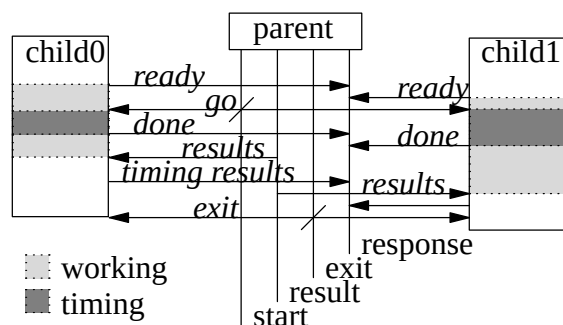
The atomicity guarantees are provided by having each client communicate synchronization states in one-byte messages. For example, the signals from the master to each child are one-byte messages, so each child only reads a single byte from the pipe. Similarly, the responses from the children back to the master are also one-byte messages. In this way no child can receive partial messages, and no message can be interleaved with any other message.

However, using this design means that we need to have a separate pipe for each *barrier* in the process, so

<sup>1</sup>This was discovered by someone who naively attempted to parallelize `lmbench2` in this fashion, and I received a note from the dismayed developer describing the failed experiment.

the master uses three pipes to send messages to the children, namely: *start\_signal*, *result\_signal*, and *exit\_signal*. If a single pipe was used for all three barrier events, then it is possible for a child to miss a signal, or if the signal is encoded into the message, then it is possible for a child to infinite loop pulling a signal off the pipe, recognizing that it has already received that signal so that it needs to push it back into the pipe, and then then re-receiving the same message it just re-sent.

However, all children share a single pipe to send data back to the master process. Usually the messages on this pipe are single-byte signals, such as *ready* or *done*. However, the timing data results need to be sent from the children to the master and they are (much) larger than a single-byte message. In this case, the timing harness sends a single-byte message on the *result\_signal* channel, which can be received by at most one child process. This child then knows that it has sole ownership of the response pipe, and it writes its entire set of timing results to this pipe. Once the master has received all of the timing results from a single child, it sends the next one-byte message on the *result\_signal* channel to gather the next set of timing results.



**Figure 1. Control signals**

The design of the signals is shown in Figure 1.

### 3.6. Benchmark initialization

By allowing the benchmark to specify an initialization routine that is run in the child processes, the new timing harness allows benchmarks to do either or both global initializations that are shared by all children and specific per-child initializations that are done independently by each child. Global initialization is done in the master process before the *benchmp* harness is called, so the state is preserved across the *fork* operations. Per-child initialization is done inside the *benchmp* harness by the optional initialization routine and is done after the *fork* operation.

Similarly, each benchmark is allowed to specify a cleanup routine that is run by the child processes just before exiting. This allows the benchmark routines to

release any resources that they may have used during the benchmark. Most system resources would be automatically released on process exit, such as file descriptors and shared memory segments, but some resources such as temporary files might need to be explicitly released by the benchmark.

### 3.7. Scheduler transients

Particularly on multi-processor systems, side-effects of process migration can dramatically affect program runtimes. For example, if the processes are all initially assigned to the same processor as the parent process, and the timing is done before the scheduler migrates the processes to other available processors, then the system performance will appear to be that of a uniprocessor. Similarly, if the scheduler is over-enthusiastic about re-assigning processes to processors, then performance will be worse than necessary because the processes will keep encountering cold caches and will pay exorbitant memory access costs.

The first case is a scheduler transient, and users may not want to measure such transient phenomena if their primary interest is in predicting performance for long-running programs. Conversely, that same user would be extraordinarily interested in the second phenomena. The harness was designed to allow users to specify that the benchmarked processes are run for long enough to (hopefully) get the scheduler past the transient startup phase, so it can measure the steady-state behavior.

### 3.8. Data analysis

Analyzing the data to produce representative results is a crucial step in the benchmarking process. *lmbench* generally reports the *median* result for 11 measurements. Most benchmarks report the results of a single measurement[8], an average of several results[11], or a trimmed mean[4].

Since *lmbench* is able to use timing intervals that are often smaller than a scheduler time slice when measuring single-process performance, the raw timing results are often severely skewed. Often most results cluster around a single value a small number of outliers with significantly larger values. The median is preferable to the mean when the data can be very skewed[9]. Since the timing intervals are significantly longer when the desired load is larger than a single process, the results tend not to be as badly skewed. In these cases we could use the *mean* instead, but we decide to use a uniform statistical framework, so we usually use the median.

In some instances, however, *lmbench* internally uses the *minimum* rather than the median, such as in *mh2*. In those instances, we are not trying to find the *representative* value, but rather the *minimum* value. There are only a few sources of error which could cause a the measured timing result to be shorter than the true elapsed time: the system clock is adjusted, or round-

off error in the clock resolution. The timing interval duration is set to ensure that the round-off error is bounded to 1% of the timing interval, and we blithely assume that people don't reset their system clocks while benchmarking their systems.

`lmbench` does not currently report any statistics representing measurement variation, such as the difference between the first and third quartiles. This is an enhancement under active consideration.

## 4. Interface

Unfortunately we had to move away from the macro-based timing harness used in `lmbench2` and migrate to a function-based system because the macros were too large for some C pre-processors.

---

```
typedef void (*bench_f)(iter_t iters,
                        void* cookie);
typedef void (*support_f)(void* cookie);

extern void benchmp(support_f initialize,
                  bench_f benchmark,
                  support_f cleanup,
                  int enough,
                  int parallel,
                  int warmup,
                  int repetitions,
                  void* cookie);

extern uint64 gettime();
extern uint64 get_n();
extern void nano(char* s, uint64 n);
extern void micro(char* s, uint64 n);
extern void mb(uint64 bytes);
```

---

**Figure 2. Programming interface**

Figure 2 shows the key elements of the new timing harness and result reporting interface. A brief description of the `benchmp` parameters:

### *enough*

Enough can be used to ensure that a timing interval is at least 'enough' microseconds in duration. For most benchmarks this should be zero, but some benchmarks have to run for more time due to startup effects or other transient behavior.

### *parallel*

is simply the number of instances of the benchmark that will be run in parallel on the system.

### *warmup*

can be used to force the benchmark to run for warmup microseconds before the system starts making timing measurements. Note that it is a lower bound, not a fixed value, since it is simply the time that the parent sleeps after receiving the last "ready" signal from each child (and before it sends the "go" signal to the children).

### *repetitions*

is the number of times the experiment should be repeated. The default is eleven.

### *cookie*

is a pointer that can be used by the benchmark writer to pass in configuration information, such as buffer size or other parameters needed by the inner loop. In `lmbench3` it is generally used to point to a structure containing the relevant configuration information.

`gettime` returns the median timing interval duration, while `get_n` returns the number of iterations executed during that timing interval.

`nano` and `micro` print the passed string latency followed by the latency in terms of nanoseconds and microseconds respectively. The latency is computed as `gettime()/n`, where `n` is the passed parameter. The reason `n` is passed as a parameter is because the benchmark can actually execute the operation of interest multiple times during a single iteration. For example, the memory latency benchmarks typically repeat the memory load operation a hundred times inside the loop, so the actual number of operations is  $100 \times \text{get\_n}()$ , and it is this value that should be passed to `nano` or `micro`.

`mb` reports the bandwidth in MB/s when given the total number of bytes processed during the timing interval. Note that for scalable benchmarks that process `size` bytes per iteration, the total number of bytes processed is `get_n() \times parallel \times size`.

---

```
#include "bench.h"

void
bench(iter_t iters, void* cookie)
{
    while (iters-- > 0) {
        getppid();
    }
}

int
main(int argc, char* argv[])
{
    benchmp(NULL, bench, NULL,
            0, 1, 0, TRIES, NULL);
    nano("getppid", get_n());
    return(0);
}
```

---

**Figure 3. A sample benchmark**

Figure 3 shows a sample benchmark that measures the latency of the `getppid` system call using this timing harness. Since there is no setup or cleanup needed for this benchmark, the *initialize* and *cleanup* parameters are NULL. The *bench* routine simply calls `getppid` as many times as requested, and the rest of the